pandas 문법 한눈에 보기

7. 데이터 분석을 위한 pandas 1 · 노트북에 등장한 문법 정리

0. 라이브러리 / 기본 설정

코드	설명
`import os, numpy as np, pandas as pd`	기본 import
`from google.colab import drive` → `drive.mount("/content/drive")`	Drive 연결
`pd.options.display.float_format = "{:,.2f}".format`	숫자 표시 형식 (쉼표, 소수 2자리)
`pd.reset_option("display.float_format")`	표시 형식 초기화
`display(표)`	한 셀에서 여러 표를 보기 좋게 출력

1. Series / DataFrame 만들기

코드	설명
`pd.Series([120,150,90], name="매출")`	Series 생성 (`index=[...]`로 인덱스 지정 가능)
`pd.DataFrame({"열": [값들], ...})`	딕셔너리로 표 생성 (리스트 길이는 모두 같아야 함)
`pd.concat([s1, s2], axis=1)`	Series 두 개를 옆으로 붙여 표 만들기 (name이 열 이름)
`df["열"]` / `df[["열1","열2"]]`	열 하나는 Series, 여러 개는 DataFrame
`s[s > 80]`	조건으로 값 골라내기

사칙연산 (연산자와 메서드는 같은 결과)

연산	연산자	메서드
더하기 / 빼기	`+` `-`	`.add()` `.sub()`
곱하기 / 나누기	`*` `/`	`.mul()` `.div()` (`.truediv()`)
몫 / 나머지 / 거듭제곱	`//` `%` `**`	`.floordiv()` `.mod()` `.pow()`

2. 파일 저장 · 불러오기

코드	설명
`os.path.join(폴더, 파일명)`	경로 만들기
`os.getcwd()`	현재 작업 폴더
`os.listdir(폴더)`	폴더 안 파일 목록
`os.path.exists(경로)`	파일 존재 여부
`os.path.basename(경로)`	경로에서 파일명만 추출
`df.to_csv(경로, index=False, encoding="utf-8-sig")`	CSV 저장
`df.to_excel(경로, index=False)`	Excel 저장
`pd.read_csv(경로)`	CSV 읽기
`pd.read_csv(경로, encoding="utf-8" / "utf-8-sig" / "cp949")`	한글 인코딩 오류 시 순서대로 시도
`pd.read_excel(경로)`	Excel 읽기 (기본은 첫 시트)
`pd.read_excel(경로, sheet_name="Returns")`	시트 이름으로 읽기
`pd.ExcelFile(경로).sheet_names`	시트 이름 목록
`df.copy()`	원본과 작업용 분리

> `index=False`는 인덱스를 파일에 열로 저장하지 않겠다는 뜻. 파일명만 쓰면 `/content`에서 찾기 때문에 Drive 파일은 전체 경로를 써야 한다 (`FileNotFoundError` 주의).

3. 데이터 구조 확인

코드	설명

`df.head(n)` / `df.tail(n)`	앞 / 뒤 n개 행 (기본 5)
`df.sample(n, random_state=42)`	무작위 n개 행 (random_state로 결과 고정)
`df.shape`	(행, 열) 개수
`df.ndim` / `df.size`	차원 수 / 전체 칸 수
`len(df)` / `len(df.columns)`	행 수 / 열 수
`df.columns`, `df.columns.tolist()`, `list(df.columns)`	열 이름
`df.index`	인덱스
`df.dtypes` / `df["열"].dtype`	자료형
`df.info()`	행·열 수, 자료형, 비어 있지 않은 값 개수, 메모리
`s.ndim`, `s.shape`, `s.size`, `s.dtype`	Series 속성

4. 결측치 · 중복

코드	설명
`df.isna()` / `df.isnull()`	빈 칸이면 True (둘은 동일)
`df.notna()`	값이 있으면 True
`df.isna().sum()`	열별 결측 개수
`df.isna().sum().sum()`	전체 결측 개수
`df.isna().mean() * 100`	열별 결측 비율(%)
`missing[missing > 0]`	결측이 있는 열만 선택
`df.fillna(0)`	값 하나로 채우기
`df["열"].fillna(df["열"].median())`	중앙값(대표값)으로 채우기
`df.fillna({"수량": 0, "라인": "미입력"})`	열마다 다르게 채우기
`df.dropna()`	하나라도 NaN인 행 삭제
`df.dropna(subset=["금액"])`	특정 열 기준 삭제
`df.dropna(how="all")`	모두 NaN인 행만 삭제
`df[df["금액"].notna()]`	값이 있는 행만 선택
`df.duplicated().sum()`	중복 행 개수
`df.drop_duplicates()`	중복 행 제거 (처음 것 유지)
`df.drop_duplicates("열", keep="last")`	해당 열 기준으로 마지막 행 유지
`df["열"].nunique()`	서로 다른 값의 가짓수

> `fillna` 와 `dropna` 는 원본을 바꾸지 않으므로 다시 대입해야 한다. 예: `df["수량"] = df["수량"].fillna(0)`, `df = df.dropna(subset=["금액"])`

NaN이 있을 때의 평균

코드	결과
`np.array(v).mean()`	`nan`
`np.nanmean(v)`	NaN을 빼고 계산
`pd.Series(v).mean()`	NaN을 빼고 계산 (기본 `skipna=True`)
`pd.Series(v).mean(skipna=False)`	`nan`

5. 요약 통계

코드	설명
`df.describe()`	숫자 열 요약 (count, mean, std, min, 25%, 50%, 75%, max)
`df.describe(include="object")`	문자열 열 요약 (count, unique, top, freq)
`df.describe(include="all")`	전체 열 요약

`df.describe().round(2)`	소수 둘째 자리 반올림

수치형 / 범주형 메서드

구분	코드
개수	`count()` (비어 있지 않은 수)
합계	`sum()`, `sum(skipna=False)`
평균 / 중앙값	`mean()`, `median()`
최대 / 최소	`max()`, `min()`
최대·최소의 인덱스	`idxmax()`, `idxmin()`
사분위수	`quantile(0.25)`, `quantile(0.75)` (`quantile(0.5)`는 중앙값)
절댓값	`abs(x)` (값 하나), `s.abs()` (열 전체)
종류 수 / 목록	`nunique()`, `unique()`
최빈값	`mode()[0]`
값별 개수 / 비율	`value_counts()`, `value_counts(normalize=True)`
가장 많은 값	`value_counts().idxmax()`
조건에 맞는 개수	`(df["매출"] >= 1000000).sum()` 또는 `len(df[조건])`

개수 세기 비교

코드	세는 것	결측
`len(df)`	전체 행 수	포함
`df["열"].count()`	비어 있지 않은 값 수	제외
`(조건).sum()`	조건이 True인 행 수	제외

6. 문자열 .str

코드	설명
`.str.strip()`	앞뒤 공백 제거
`.str.upper()` / `.str.lower()`	대문자 / 소문자
`.str.replace("-", "", regex=False)`	문자 바꾸기 (`regex=False`는 입력한 문자 그대로)
`.str.len()`	글자 수
`.str.contains("모터")`	포함 여부
`.str.startswith("MT")` / `.str.endswith("01")`	시작 / 끝 확인
`.str.split("-")` / `.str.split("-", expand=True)`	나누기 (expand=True는 열로 분리)
`.str[:2]`, `.str[1:]`	슬라이싱
`.str.count("-")`	특정 문자 개수

7. 자료형 변환

코드	설명
`.astype(int / float / str)`	지정한 자료형으로 변환
`pd.to_numeric(s)`	숫자로 변환 (불가능한 값이 있으면 오류)
`pd.to_numeric(s, errors="coerce")`	변환 불가능한 값은 `NaN`
`pd.to_datetime(s)`	날짜형으로 변환
`pd.to_datetime(s, errors="coerce")`	변환 불가능한 값은 `NaT`

쉼표가 있는 문자 숫자 정리

```
df["가격"] = df["가격"].str.replace(",", "", regex=False).astype(int)
```

> 결측이 있으면 `fillna(0).astype(int)` 처럼 먼저 결측을 처리한 뒤 정수로 변환한다.

자료형	의미
`int64` / `float64`	정수 / 실수
`object`, `str`	문자열
`bool`	참·거짓
`datetime64[ns]`	날짜·시간

8. 날짜 .dt

코드	설명
`.dt.year` / `.dt.month` / `.dt.day`	연 / 월 / 일 (속성, 괄호 없음)
`.dt.hour` / `.dt.minute`	시 / 분
`.dt.dayofweek`	요일 번호 (월=0 … 일=6)
`.dt.day_name()`	요일 이름 (메서드, 괄호 있음)
`.dt.to_period("M")`	월 단위 기간
`df[df["주문일"] >= "2026-04-01"]`	날짜 비교
`배송일 - 주문일`	날짜 차이 → `Timedelta`
`.dt.days`	기간의 일수 부분
`.dt.seconds`	일수를 뺀 나머지 초
`.dt.total_seconds()`	전체 기간을 초로 (`/60` 분, `/3600` 시간)
`주문일 + pd.Timedelta(days=7)`	날짜 더하기 (`weeks=`, `hours=`, `minutes=` 도 가능)
`.dt.strftime("%Y-%m-%d")`	날짜 → 문자열

strftime 기호

기호	의미	기호	의미
`%Y`	연도 4자리	`%H`	시 (24시간)
`%m`	월	`%M`	분
`%d`	일	`%S`	초

> `%m` (월)과 `%M` (분)은 대소문자로 구분한다. `strftime` 결과는 문자열이므로 날짜 계산은 원래 날짜형으로 한다. `NaN` 은 숫자의 결측, `NaT` 는 날짜의 결측.

9. 기타 자주 쓰는 패턴

코드	설명
`round(x, 2)`	파이썬 내장 반올림
`sum(s)`	파이썬 내장 합계 (`s.sum()` 과 같은 결과)
`.tolist()`	Series를 리스트로 변환
`type(obj)`	자료형 확인
`df["새열"] = 계산식`	새 열 추가